

Agent Mesh Protocol — v1.2

MQTT mesh for human/agent → agent job dispatch. Broker root: `agents` .

Agent Mesh Protocol · version 1.2 · reference implementation: OpenClaw

agent-mesh-bridge plugin · generated 2026-08-24

Delivery is **push end to end**. Nothing on a delivery path polls: the broker pushes over a persistent MQTT session, the plugin pushes the job into an executor subagent the moment it arrives, results are pushed back at QoS 1 (retained), and the web panel receives Server-Sent Events. The only periodic timers left are a supervisory watchdog and a slow filesystem reconciler, neither of which carries a message.

Registry (retained)

Topic	Direction	Payload
<code>agents/registry/<agentId>/profile</code>	agent → all (retained)	<code>{ agentId, displayName, status, protocolVersion, capabilities[], ownerPolicy }</code>
<code>agents/registry/<agentId>/status</code>	agent → all (retained + LWT)	<code>{ status: online offline }</code>

`ownerPolicy` is `{ required: bool, verified: bool }` — it tells you what this deployment actually enforces, so you never have to infer enforcement from the version number.

Commands

Topic	Direction	Notes
<code>agents/commands/<agentId>/invoke</code>	you → agent	<code>{ service, args, requestedBy, jobId? }</code> — requestedBy is REQUIRED and is now enforced
<code>agents/commands/<agentId>/query</code>	you → agent	<code>{}</code> lists services; <code>{ jobId }</code> checks state. Reply on <code>.../query/reply</code>
<code>agents/commands/<agentId>/cancel</code>	you → agent	<code>{ jobId, requestedBy }</code> . Reply as a terminal <code>cancelled</code> result
<code>agents/commands/<agentId>/config</code>	you → agent	service CRUD. Reply on <code>.../config/reply</code>

Capabilities are data, not code

The bridge is **capability-agnostic**. It contains no service name — not `code.review` , not anything else. A capability is a JSON entry in `services.json` :

```
{ "service": "code.review",
  "description": "...",
  "requestSchema": { "repo": "string (owner/name)", "pr": "number" },
  "handler": "session",
  "prompt": "Review pull request {{pr}} in {{repo}}. ..." }
```

`invoke` names a `service`; the bridge looks it up, renders its `prompt` with `{{jobId}}`, `{{requestedBy}}` and each key of `args`, and hands the result to an executor. That is the entire coupling. Everything domain-specific lives in the prompt template.

Capabilities can be added, updated and removed at runtime over the `config` command (or the panel) with no restart and no code change — the retained profile republishes automatically. The catalog shipped here is an **example**; replace it wholesale for a different domain and the transport is unchanged.

Deployment values: `${ENV_VAR}` in prompts

A capability definition should stay portable, but real prompts need deployment-specific values — Slack ids, channel names, internal repo paths. Put them in the environment and reference them from the prompt:

```
"prompt": "Review PR {{pr}} in {{repo}}. DM the summary to ${SLACK_REVIEW_RECIPIENTS}."
```

Two properties make this safe, and both are deliberate:

- **Resolved at dispatch, never in the catalog.** The capability list is published to the *retained* profile topic. Resolving there would broadcast every deployment value to anyone subscribed to the registry. On the wire the prompt stays a template; substitution happens only on the way to the executor.
- **Environment first, then arguments.** `${...}` is expanded *before* `{{args}}`, so a value arriving in an `invoke` argument is never itself expanded. Were the order reversed, any caller could pass `"${SOME_SECRET}"` as an argument and have the bridge read it back — turning every `invoke` into an environment read.

An unset variable substitutes empty and logs a warning; it never leaks the literal `${NAME}` into the executor's instructions.

Note the boundary this respects: env vars supply *identifiers and configuration*, not credentials for the mesh to use. Authentication still belongs to the executor.

Credentials & scope boundary

The bridge carries jobs, not credentials. It holds exactly two secrets: the broker username/password, and the optional `web.auth` panel token. It has no GitHub, cloud or third-party authentication of any kind, and no code path that acquires or forwards one.

Service credentials belong to the **executor**, which authenticates with the host's own configured identity (for example an already-authenticated `gh` CLI). This follows directly from "transport in the framework, logic in the agent": an agent that needs to read a private repository is exercising its own authority, not authority delegated through the mesh.

Never place tokens in `invoke` args. Payloads are readable by every subscriber to that topic, and results are **retained on the broker**, so a secret published once persists until it is explicitly overwritten.

Durability

The bridge holds one **persistent session**: `clean: false`, QoS 1 on every subscription, 30-second keepalive, 5-second auto-reconnect, LWT for presence. The connection is outbound only, so it works behind NAT.

Durability depends on the **client id being stable across restarts**. It is derived from hostname + install path — never from the process id. This matters more than it looks: with a changing id, every restart is a *new* MQTT session, so the broker's queued QoS-1 messages stay orphaned with the dead session and any `invoke` published while the gateway was down is lost silently. It also leaked one abandoned session per restart, since MQTT 3.1.1 has no session expiry.

- An `invoke` published while the bridge is down is **queued by the broker and delivered on reconnect**.
- If you genuinely run two gateways from the same install directory, set `broker.clientId` explicitly on one of them. Session durability and multi-instance safety are in tension and the operator has to choose. A collision manifests as a kick-loop, which the bridge detects and logs explicitly rather than letting it look like flaky networking.
- Set `broker.protocolVersion: 5` to get a bounded `sessionExpirySeconds` (default 24h), so a decommissioned deployment stops accumulating queued messages forever.

In-memory job tracking (`activeJobs`, history) does not survive a restart. Retained results replay on resubscribe, so completed jobs reappear; jobs in flight across a restart are no longer watched and rely on the executor publishing its own retained result.

Identity & trust model

MQTT delivers **topic + payload only** to subscribers. No username, no client id — the publisher's broker identity never travels with the message. Therefore:

- `owner` is **self-declared**: whatever string the sender puts in `requestedBy`.
- With the default configuration, that string is *required* but not *authenticated*. Anyone with broker credentials can claim any owner scope by forging `requestedBy`.
- Privacy/isolation is therefore **conventional, not enforced**, unless you enable verification below.

Optional owner verification (`mesh.verifyOwner`)

The bridge can verify `requestedBy` against a broker-injected `client_username`, using an EMQX rule-engine payload enrichment. When enabled it **fails closed**: an invoke that arrives without `client_username` is rejected, and one whose `requestedBy` disagrees with the broker identity is rejected with the mismatch reported.

This is **off by default** because it requires broker-side setup first. Wire the EMQX rule, then turn it on. Until you do, `ownerPolicy.verified` is `false` in the profile and clients should treat owner scoping as a convention.

Stronger isolation remains a broker-side concern: EMQX ACLs (`subscribe jobs/${username}/#`, `publish commands/+/invoke`) enforce it at the boundary rather than in the agent.

Owner

`owner` scopes job traffic so each requester only receives their own jobs:

- `owner` = `requestedBy` from the invoke payload, lowercased, restricted to `[a-z0-9_-]` (anything else → `-`), trimmed.
- **Set `requestedBy` to your stable username** (e.g. `mohanad`, `ci-github`, `kg`). It's the same value for invoke and for your subscription filter.
- **Omitting it is rejected.** The job is refused and a `type: "error"` result is published to `jobs/public/<jobId>/result` so the failure is never a silent drop. Previously such jobs were quietly routed to `public`, where the sender's own subscription filter would never see them.
- Set `mesh.requireOwner: false` to restore the lenient behaviour, which accepts the invoke, logs a warning, and scopes it to `public`.
- Anyone subscribing the flat `jobs/#` still sees everything (the agent's own panel does this) — clients shouldn't.

Job lifecycle topics

```
agents/jobs/<owner>/<jobId>/events    milestones: accepted, started, diff-fetched, analyzing, result
agents/jobs/<owner>/<jobId>/result    terminal payload, RETAINED, QoS 1
```

Every result carries `jobId`, `owner`, `ts`, and `type`. Known types: `review`, `already_reviewed`, `error`, `duplicate`, `cancelled`, and the generic `result`. The list is open — treat an unrecognised `type` as terminal.

These guarantees are enforced by the bridge, not by the executor. Any publish to a job topic through the `mqtt_publish` tool is normalised on the way out: `jobId`, `owner` and `ts` are injected from the topic if absent, `type` is defaulted, and **retain is forced true on result topics**. Conformance no longer depends on the executor remembering to set the flag — a forgotten `retain: true` used to mean late subscribers saw nothing at all.

Events are QoS 1 and deliberately **not** retained: a subscriber joining mid-job sees the retained result but not earlier milestones.

Job topics are **always owner-scoped**. There is no unscoped `jobs/<jobId>/...` form: a result belongs to exactly one owner and is published to exactly one topic.

Cancellation

`cancel` is **cooperative and terminal at the mesh boundary**:

1. `cancel_acknowledged` is published to the job's event stream.
2. A retained `type: "cancelled"` result is published, so listeners are never left waiting on a job that will never report.
3. Any later publish from the executor for that job is **suppressed**, keeping the client's view consistent with the acknowledgement.
4. The executor's session is dropped on a best-effort basis to stop it consuming budget.

What it does **not** guarantee is immediate termination of in-flight work. The plugin runtime exposes `run`, `waitForRun`, `getSession` and `deleteSession` — there is no abort primitive — so an executor mid-tool-call may run to completion internally. Its output is discarded rather than published. Treat `cancelled` as "no further traffic and no result will be honoured", not as "the compute stopped".

Execution & liveness

Jobs are pushed straight into an isolated executor subagent at arrival — no heartbeat, no queue, so a busy or failing main session cannot swallow a job. If the runtime has no subagent API, the bridge falls back to a system-event enqueue plus heartbeat wake, which is the only pull-shaped path that remains, and only on older runtimes.

A supervisory watchdog sweeps every 60 seconds. Its liveness signal is **whether the executor's run has settled**, not whether the executor has been chatty:

- Run still in flight → alive by definition, never re-dispatched.
- Run settled without a published result → re-dispatched, up to 2 times, then a terminal `error` result.
- Fallback path with no run handle → falls back to a 5-minute silence heuristic.
- Any job exceeding `mesh.maxJobDurationMs` (default 30 min) fails with a terminal error regardless of liveness.

The previous heuristic re-dispatched any job that worked *silently* for five minutes, which a large PR review does routinely — causing duplicate execution of exactly the most expensive jobs. Executors are still asked to publish a milestone every two minutes, but correctness no longer depends on their doing so.

What you should subscribe to

As a job sender with username `mohanad` :

```
agents/jobs/mohanad/#      ← your jobs: events + results (recommended)
agents/jobs/mohanad/+/result ← only final results, no progress noise
agents/commands/conan/query/reply ← if you use query
```

Do NOT subscribe `agents/jobs/#` — that's the pre-1.1 firehose of everyone's jobs.

As the agent (conan) itself: the four `commands/conan/*` topics + `agents/jobs/#` (full history for the web panel).

Example flow

1. Publish `agents/commands/conan/invoke` :

```
{ "service": "code.review", "requestedBy": "mohanad",
  "args": { "repo": "owner/repo", "pr": 1234 } }
```

2. Listen on `agents/jobs/mohanad/#` .
3. Events arrive as `.../jobs/mohanad/<jobId>/events` ; the final review (retained) lands on `.../jobs/mohanad/<jobId>/result` .

Omit `requestedBy` and step 1 is rejected — you'll find the error on `agents/jobs/public/<jobId>/result`, not on your own scope.

Web control panel

A front end **for** this plugin; no MQTT in the browser. Served on `127.0.0.1:8765` by default.

Route	Purpose
GET <code>/api/profile</code>	catalog + connection state
GET <code>/api/status</code>	broker stats (uptime, rx/tx, reconnects)
GET <code>/api/jobs</code>	active jobs + recent history
GET <code>/api/events</code>	SSE stream of job/status/profile changes
POST <code>/api/invoke</code>	{ <code>service</code> , <code>args</code> , <code>requestedBy</code> , <code>jobId?</code> }
POST <code>/api/cancel</code>	{ <code>jobId</code> , <code>requestedBy</code> }
POST <code>/api/config</code>	service CRUD

The panel consumes the SSE stream and only falls back to polling if the stream drops. Set `web.auth` to require a bearer token — it is accepted as an `Authorization` header, or as `?token=` for the SSE stream, since `EventSource` cannot set headers. This setting was previously declared but never enforced; it is now enforced on every route.

Changes v1.1 → v1.2

Protocol:

- `requestedBy` is enforced, not merely documented as required. Rejections are published rather than silently rerouted to `public`.
- Optional broker-identity verification (`mesh.verifyOwner`), fail-closed, off by default.
- `ownerPolicy` advertised in the profile and in `query` replies, so clients can read the deployment's actual enforcement instead of inferring it.
- `cancelled` added as a terminal result type; cancel now always produces a terminal result.

Conformance fixes:

- Result payload shape and the retained flag are guaranteed by the bridge rather than by executor prompt-following.

Removed:

- The v1.0 compatibility layer is gone: no unscoped `jobs/<jobId>/result` mirror, no legacy request topic, and no `topics` config block. It existed only to carry a single deployment through the 1.0 → 1.1 migration, and it forced a service name into the transport layer.
- With it goes the last hardcoded capability. The bridge no longer contains the string `code.review`, or any other service name.

Durability & delivery:

- Client id is stable across restarts, so `clean: false` actually persists the session and invokes published during downtime survive.
- Optional MQTT 5 with a bounded session expiry.
- Web panel moved from a 2.5-second poll to Server-Sent Events.
- `services.json` changes detected via `fs.watch` instead of a 30-second mtime poll.
- Watchdog liveness switched from executor chattiness to run settlement, removing duplicate execution of long silent jobs; hard per-job duration cap added.
- `web.enabled` and `web.auth` are honoured; both were previously no-ops.